incluye refinamiento

2) Time box

- Incluye sprint, sprint planning, daily, sprint review, sprint retrospective y refinamiento

3) De salida a impacto

código limpio + documentación

- El incremento permite identificar que se cumplieron las definiciones técnicas en el sprint. O sea se le suma la VALIDACIÓN de que lo que estamos entregando genera valor (impacto negocio)

output done / outcome done

¡! Nunca hay que perder de vista el obj. Final (que puede incluir varios sprint)

4) Cada artefacto tiene un obj

- Product backlog (no tiene tareas) → dele tareas → OBJ estratégico de prod
- Sprint backlog (PLAN) (tiene tareas) → objetivo del sprint
- Incremento prod → definition of done → Estandar de calidad

25/09

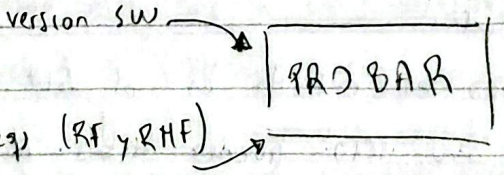
TESTING

DEFINICIÓN proceso (destructivo) cuyo propósito es encontrar defectos cuya presencia se asume (en el software PROC DE CONTROL → agilismo ágil)

testing ágil trata de solucionar esto

Históricamente había peleas entre desarrolladores y testing

→ Como cualquier proc de control, se compara una versión de un producto de SW (código) contra los requerimientos (FUNC y no FUNC)



- Se hace uso de CASOS DE PRUEBA para hacer testing sistemático

¡! Un error no reproducible no es un error

[En el PUD, el testing es un workflow más que viene después de la implementación]

(?) Ojo con los títulos (hay muchas clasificaciones / temas)

Contexto

- Tener una buena gestión de CONFIG para el aseguramiento de calidad del software

QA (calidad) ≠ TESTING (prueba calidad, se hace al final)

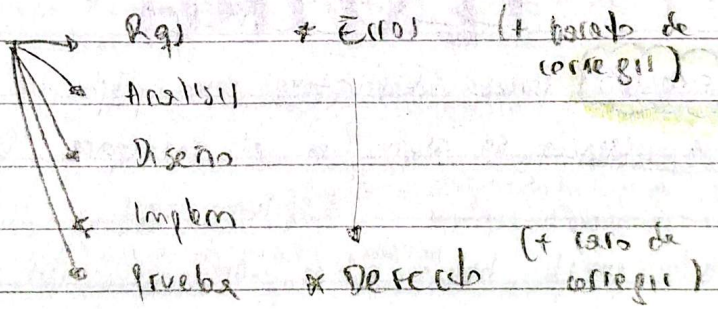
(abarca + cosas (calidad prod. y calidad proceso)
no se hace solo al final

* Testing no asegura ni garantiza la calidad. Hace lo posible para hacer visibles los defectos del sist. No puede asegurar que no hayan más defectos

Error vs Defecto

- Se diferencia de un defecto según el momento en que se encuentra.

Recordar: Proc



- El defecto se traslada de la etapa que lo introdujo. Los defectos de código son baratos de corregir, pero los defectos de requerimientos son caros.
- La falla es lo que provoca el defecto en el SW. Es un mal funcionamiento del SW. Pero pueden haber defectos

que no generen fallas (ej mala implementación del interfaz Front). es por eso que no todos los defectos son igual de graves

Severidad (para el func usual del sist)

Lo define testing → Determina la gravedad del defecto encontrado se debe establecer de antemano en el equipo la clasificación a utilizar

- Bloqueante no anda el sist
- Crítico
- Mayor no puede cumplir la func del modo normal, pero encuentro algún atajo
- Menor el obj se cumple con algún detalle
- Cosmético mala ortografía, colores

Prioridad ¿qué tan imp es para el usuario Final

Lo define el cliente → Asociado a la perspectiva del usuario. Implica cuan necesario es para el usuario, la func especificada. También se clasifica:

- Urgencia no siempre lo urgente tiene severidad crítica
- Alta
- Media
- Baja

Nos sirve para saber qué corregir primero (sobretodo si ya está en producción

Workflows de implementación
1) Hago los test unit.
2) Hago los comp
3) pruebo que anda

TDD | test driven developer - desarrollo conducido por pruebas

BUILD | comp de comp integrados

Niveles de prueba

1) Pruebas unitarias (se prueba cada comp una vez desarrollo)
Los hace el desarrollador. Se automatizan.

WORKFLOW IMPLEM.

2) Pruebas de integración / interfaz
Ver si los comp funcionan bien integrados en el build
El libro dice que lo hace testing. En la realidad testing no tiene ese WF PRUEBA

3) Pruebas de sistema / versión (pasa a)
Lo hace testing. Prueban una versión del sistema. Tiene que hacerlo ≠ de quien desarrolló para encontrar defectos

4) Pruebas de aceptación de usuario WF DESPLIEGUE

Se hace en el workflow de despliegue. Lo hace el usuario DE esta pruebas depende si se paga o no el desarrollo

En scrum se hace en la review

VALIDACION: hace lo que tiene que hacer

VERIFIC: que lo hago bien

con testing pruebo ambas cosas

Ambientes para construc software

- Desarrollo (unitarias y de integración
- Pruebas (sistema
- Pre-prod es en donde se hacen las pruebas de aceptación
- Producción

* Repositorio con toda la config necesaria (ambiente limpio)

Casos de prueba

- Paso a paso INSTRUCCIONES en base al requerimiento
- Escenario de uso único de un sistema
- Muestra un escenario de un caso de uso (ej un ejemplo)
En base a ciertos datos, ejecuto una serie de instrucciones y espero obtener ciertos resultados.
- Se generan en el mismo proceso de prueba
- Si testing genera CP en base al código, ya q podes verificar pero no validar, por eso necesita de la user (los req)

Consideraciones

- Los errores suelen estar en los límites, las excepciones (análisis de técnica valores límites)
  - puedo reducir la cant de CP sin reducir su imp para encontrar errores
- Estrategias
  - Testing caja negra ejecuto (no se cómo funciona) (sist func)
  - Testing caja blanca estática, veo el código

Se complementan. En caja negra no puedo ver si hay alg ineficientes, código malicioso (se reporta con un informe)

Revisión de código → se hace en el WF de la implementación, lo hace un par, no testing, pero puede usar algún mét de caja blanca

Ciclo de prueba

- En base a los req, diseño un conj de CP (Σ CP), y cuando llega la implementación, ejecuto los CP y voy guardando los errores encontrados
- Ciclo 0 → primera ejecución de CP

Regresión

Sin regresión → solo reviso la resolución de los errores reportados

mejor

Con regresión → pruebo todo de nuevo como si fuera ciclo 0

muy util sobretodo si hay automatización

Proceso de prueba

1) Planificación necesito los req
2) Identificación } diseño pruebas y configuro la BD con datos reales
3) Especificación }
4) Ejecución (cuando llega la versión)
5) Analisis de fallas → defectos (reporte)
6) Fin de las pruebas → deja de probar